SEMAINE 1 -- MARDI

Methodes Essentielles des Tableaux JavaScript

Programme de Renforcement DWWM -- Classe CodeWave

Objectifs de la seance
A la fin de cette seance, vous serez capable de :
Utiliser les methodes de base : push, pop, shift, unshift, splice
Maitriser les methodes fonctionnelles : forEach, map, filter, find, findIndex, reduce, sort
Appliquer le spread operator et la destructuration de tableaux
Resoudre des problemes de logique en combinant plusieurs methodes

PARTIE 1 -- Documentation & Exercices Debutants

Lisez attentivement la documentation de chaque methode, testez les exemples dans votre console, puis realisez les exercices correspondants.

1.1 push(), pop(), shift(), unshift(), splice()

Ces methodes modifient directement le tableau (mutation). Elles sont indispensables pour gerer des listes dynamiques.

JavaScript

```javascript
const fruits = ['pomme', 'banane'];

fruits.push('cerise');        // ajoute a la fin  =>
['pomme','banane','cerise']
const x = fruits.pop();       // retire la fin    => x = 'cerise'
const y = fruits.shift();     // retire le debut  => y = 'pomme'
fruits.unshift('fraise');     // ajoute au debut  => ['fraise','banane']

// splice(index, nbSupprimes, ...elementsAjoutes)
fruits.splice(1, 0, 'kiwi'); // insere 'kiwi' en index 1 sans rien supprimer
// => ['fraise','kiwi','banane']
fruits.splice(0, 1);          // supprime 1 element a l'index 0
// => ['kiwi','banane']
```

Exercice 1 -- Ma premiere liste	Debutant	10 min

Partez d'un tableau vide et suivez les etapes :

1. Creez un tableau taches = []
2. Ajoutez 'Coder', 'Tester', 'Deployer' a la fin avec push()
3. Ajoutez 'Analyser' au debut avec unshift()
4. Retirez la derniere tache avec pop() -- affichez la valeur retiree
5. Retirez la premiere tache avec shift() -- affichez la valeur retiree
6. Inserez 'Documenter' entre 'Coder' et 'Tester' avec splice()
7. Remplacez 'Tester' par 'Revue de code' avec splice()
8. Affichez le tableau final -- vous devez obtenir : ['Coder', 'Documenter', 'Revue de code', 'Deployer']

1.2 forEach() -- Parcourir

forEach() execute une fonction sur chaque element. Elle ne retourne rien (undefined). Utilisez-la uniquement pour les effets de bord (console.log, pousser dans un autre tableau...).

JavaScript
```
const nombres = [10, 20, 30];

nombres.forEach((valeur, index) => {
    console.log(`[${index}] => ${valeur}`);
});
// [0] => 10
// [1] => 20
// [2] => 30
```

Exercice 2 -- Affichage d'une liste	Debutant	8 min

JavaScript
```
const prenoms = ['Alice', 'Bob', 'Clara', 'David', 'Eva'];
```

1. Utilisez forEach() pour afficher chaque prenom avec son rang : '1. Alice', '2. Bob'...
2. Utilisez forEach() pour construire un nouveau tableau longueurs contenant le nb de lettres de chaque prenom, puis affichez-le
3. Utilisez forEach() pour afficher uniquement les prenoms dont la longueur est superieure a 3 lettres

1.3 map() -- Transformer

map() cree un NOUVEAU tableau transforme. Le tableau d'origine reste intact. Retourne toujours un tableau de meme taille.

JavaScript
```
const prix = [10, 25, 8];

const prixTTC = prix.map(p => p * 1.20);  // [12, 30, 9.6]
```

```
const etiquettes = prix.map(p => `${p} EUR`); // ['10 EUR','25 EUR','8 EUR']

console.log(prix); // [10, 25, 8] — inchange !
```

Exercice 3 -- Transformation de donnees	Debutant	8 min

```JavaScript
const temperatures = [0, 15, 22, 35, -5, 100];
```

1. Convertissez en Fahrenheit avec map() (formule : F = C * 9/5 + 32)
2. Creez un tableau de descriptions : '>= 30 => Chaud', '< 0 => Froid', sinon => 'Tempere'
3. Creez un tableau d'objets { celsius: val, statut: '...' } en combinant les deux idees precedentes

1.4 filter() -- Filtrer

filter() cree un nouveau tableau avec seulement les elements pour lesquels la fonction retourne true.

```JavaScript
const ages = [12, 18, 25, 16, 30, 14];

const majeurs = ages.filter(a => a >= 18);  // [18, 25, 30]
const pairs   = ages.filter(a => a % 2 === 0); // [12, 18, 16, 30, 14]
```

Exercice 4 -- Filtrage simple	Debutant	8 min

```JavaScript
const mots =
['chat','cheval','chien','lion','chameau','cobra','loup','chevre'];
```

1. Filtrez uniquement les mots qui commencent par 'ch' (utilisez startsWith)
2. Filtrez les mots de plus de 5 lettres
3. Combinez les deux : mots qui commencent par 'ch' ET ont plus de 5 lettres

1.5 find() et findIndex() -- Rechercher

find() retourne le premier element correspondant, ou undefined. findIndex() retourne son index, ou -1 si introuvable.

```JavaScript
const users = [
    { id: 1, nom: 'Alice', actif: true  },
```

```
    { id: 2, nom: 'Bob',   actif: false },
    { id: 3, nom: 'Clara', actif: true  },
];

const user = users.find(u => u.id === 2);       // { id:2, nom:'Bob',
actif:false }
const idx  = users.findIndex(u => u.nom === 'Clara'); // 2
const absent = users.find(u => u.id === 99);    // undefined
```

Exercice 5 -- Recherche dans un catalogue	Debutant	8 min

```javascript
const catalogue = [
    { ref: 'A01', nom: 'Stylo bille',  prix: 1.20 },
    { ref: 'A02', nom: 'Cahier A4',    prix: 3.50 },
    { ref: 'A03', nom: 'Surligneur',   prix: 2.10 },
    { ref: 'A04', nom: 'Post-it',      prix: 3.80 },
    { ref: 'A05', nom: 'Ciseaux',      prix: 6.30 },
];
```

1. Trouvez le produit de reference 'A03' avec find()
2. Trouvez l'index du 'Cahier A4' avec findIndex()
3. Cherchez la ref 'A99' et affichez un message adapte si elle n'existe pas (gerez le undefined)

1.6 reduce() -- Agreger

reduce() transforme un tableau en une valeur unique : somme, objet, tableau... La methode la plus puissante.

```javascript
// reduce(callback, valeurInitiale)
// callback recoit : (accumulateur, elementCourant, index, tableau)

const nb = [1, 2, 3, 4, 5];

const somme = nb.reduce((acc, n) => acc + n, 0);  // 15
const max   = nb.reduce((acc, n) => n > acc ? n : acc, nb[0]); // 5

// Compter les occurrences
const animaux = ['chat','chien','chat','oiseau','chat'];
const compte = animaux.reduce((acc, a) => {
    acc[a] = (acc[a] || 0) + 1;
    return acc;
```

```
}, {});
// { chat: 3, chien: 1, oiseau: 1 }
```

Exercice 6 -- Premiers calculs avec reduce	Debutant	10 min

```javascript
const notes = [14, 8, 17, 11, 15, 9, 18, 12];
```

1. Calculez la somme de toutes les notes
2. Calculez la moyenne (somme / nb de notes)
3. Trouvez la note la plus haute avec reduce() (sans utiliser Math.max)
4. Comptez combien de notes sont au-dessus de la moyenne que vous venez de calculer

1.7 sort() -- Trier

sort() trie le tableau EN PLACE (mutation). Sans argument, tri alphabetique. Pour les nombres, passez une fonction comparateur.

```javascript
// ATTENTION : sort() sans argument = tri alphabetique des chiffres aussi !
[10, 1, 21, 2].sort();                // [1, 10, 2, 21]  -- FAUX !
[10, 1, 21, 2].sort((a,b) => a - b);  // [1, 2, 10, 21] -- correct
[10, 1, 21, 2].sort((a,b) => b - a);  // [21, 10, 2, 1] -- decroissant

// Trier des objets
const produits = [{ nom:'B', prix:5 }, { nom:'A', prix:2 }];
produits.sort((a,b) => a.prix - b.prix);          // par prix croissant
produits.sort((a,b) => a.nom.localeCompare(b.nom)); // alphabetique
```

Exercice 7 -- Tris simples	Debutant	8 min

```javascript
const scores = [45, 12, 78, 3, 99, 56, 23, 67];
const noms   = ['Zoe','Alice','Marc','Bob','Yasmine','Chloe'];
```

1. Triez scores en ordre croissant SANS muter le tableau d'origine (utilisez le spread [...scores])
2. Triez scores en ordre decroissant de la meme facon
3. Triez noms par ordre alphabetique avec localeCompare()
4. Affichez les 3 meilleurs scores (trier puis slice)

1.8 Spread operator et Destructuration

Le spread operator (...) decompose un tableau. La destructuration extrait ses valeurs dans des variables nommees.

```javascript
// ---- Spread ----
const a = [1,2,3];
const b = [4,5,6];
const fusion = [...a, ...b];          // [1,2,3,4,5,6]
const copie  = [...a];                // copie independante
const max    = Math.max(...a);        // 3

// ---- Destructuration ----
const [x, y, z] = [10, 20, 30];
const [premier, ...reste] = [1,2,3,4]; // premier=1, reste=[2,3,4]

// Echanger deux variables en une ligne
let p = 1, q = 2;
[p, q] = [q, p];                      // p=2, q=1

// Valeurs par defaut
const [val, unite = 'kg'] = [42];     // val=42, unite='kg'
```

Exercice 8 -- Spread et Destructuration	Debutant	10 min

Partie A -- Spread

1. Creez fruits=['pomme','poire'] et legumes=['carotte','tomate'], fusionnez-les dans aliments avec spread
2. Copiez aliments dans copiePanier, ajoutez 'cerise' a copiePanier, verifiez qu'aliments est intact
3. Trouvez le max et le min de [3,1,4,1,5,9,2,6] avec Math.max/min et le spread

Partie B -- Destructuration

1. Destructurez const couleurs=['rouge','vert','bleu','jaune'] pour obtenir : premiere, deuxieme, et un tableau autresCouleurs
2. Echangez les valeurs de x=10 et y=20 en une seule ligne grace a la destructuration
3. Destructurez const data=[42] avec valeur par defaut : const [val, unite='kg'] = data -- affichez les deux

PARTIE 2 -- Challenges Intermediaires

Consigne

Ces challenges ne sont pas des exercices guidés.
On vous donne une situation et un resultat attendu : c'est a vous de trouver la logique.
Vous pouvez utiliser n'importe quelle methode vue en partie 1.
Lisez bien l'exemple d'entree/sortie avant d'ecrire une seule ligne.

Exercice I1 -- Nettoyeur de tableau | Intermediaire | 15 min

On vous donne une fonction a implementer :

```javascript
function nettoyer(tableau) {
    // Votre code ici
}
```

La fonction doit retourner un nouveau tableau ou :

- les doublons sont supprimes (chaque valeur n'apparait qu'une fois)
- les valeurs falsy (0, '', false, null, undefined) sont retirees
- le resultat est trie par ordre croissant

Code	Resultat attendu
`nettoyer([3,1,2,1,3,0,'',5,null,2])`	`=> [1, 2, 3, 5]`
`nettoyer([false,7,7,'',8,undefined,8])`	`=> [7, 8]`
`nettoyer([0,0,0])`	`=> []`

Indice
Pensez a filter() pour eliminer les falsy et les doublons.
Pour les doublons : indexOf() ou includ() peuvent vous aider.

Exercice I2 -- Rotation de tableau | Intermediaire | 15 min

Implementez la fonction rotate(tableau, n) qui decale tous les elements de n positions vers la droite. Les elements qui 'depassent' a droite reviennent au debut.

```javascript
function rotate(tableau, n) {
    // Votre code ici
```

Code	Resultat attendu

`rotate([1,2,3,4,5], 2)`	`=> [4, 5, 1, 2, 3]`
`rotate([1,2,3,4,5], 0)`	`=> [1, 2, 3, 4, 5]`
`rotate(['a','b','c'], 1)`	`=> ['c', 'a', 'b']`
`rotate([1,2,3,4,5], 7)`	`=> [4, 5, 1, 2, 3]  (7 % 5 = 2)`

Indice
Pensez au modulo (%) pour gerer les n > longueur du tableau.
slice() et spread peuvent vous faire gagner du temps.

Exercice I3 -- Aplatisseur

Intermediaire | 20 min

Implementez flatten(tableau) qui transforme un tableau de tableaux imbriques en un tableau plat, quel que soit le niveau d'imbrication.

JavaScript
```
function flatten(tableau) {
    // Votre code ici -- sans utiliser flat() ni flatMap()
}
```

Code	Resultat attendu
`flatten([1,[2,3],[4,[5,6]]])`	`=> [1, 2, 3, 4, 5, 6]`
`flatten([[1,[2]],[[[3]],4]])`	`=> [1, 2, 3, 4]`
`flatten([1,2,3])`	`=> [1, 2, 3]`

Indice
Pour chaque element : est-ce un tableau ? Si oui, recurse. Sinon, pousse.
reduce() + recursivite = combinaison gagnante ici.

Exercice I4 -- Intersection et Difference

Intermediaire | 20 min

Implementez deux fonctions :

- intersection(a, b) : retourne les elements presents dans les deux tableaux (sans doublons)
- difference(a, b) : retourne les elements presents dans a mais pas dans b

JavaScript
```
function intersection(a, b) { /* ... */ }
function difference(a, b)   { /* ... */ }
```

Code	Resultat attendu
`intersection([1,2,3,4], [2,4,6])`	`=> [2, 4]`
`intersection([1,2,3], [4,5,6])`	`=> []`
`difference([1,2,3,4], [2,4])`	`=> [1, 3]`
`difference([1,2,3], [1,2,3,4])`	`=> []`

Exercice I5 -- Grouper par cle

Intermediaire | 20 min

Implementez groupBy(tableau, cle) qui regroupe les objets d'un tableau selon la valeur d'une propriete.

JavaScript
```javascript
function groupBy(tableau, cle) {
    // retourne un objet { valeurDeLaCle: [elements], ... }
}
```

JavaScript
```javascript
const produits = [
    { nom:'Stylo',    categorie:'ecriture', prix: 1.5 },
    { nom:'Cahier',   categorie:'papier',   prix: 3.5 },
    { nom:'Feutre',   categorie:'ecriture', prix: 2.0 },
    { nom:'Post-it',  categorie:'papier',   prix: 3.8 },
    { nom:'Ciseaux',  categorie:'bureau',   prix: 6.5 },
```

Resultat attendu

```
{
  ecriture: [ {Stylo...}, {Feutre...} ],
  papier:   [ {Cahier...}, {Post-it...} ],
  bureau:   [ {Ciseaux...} ]
}
```

BONUS : apres avoir implemente groupBy, utilisez-la pour afficher le prix moyen par categorie.

PARTIE 3 -- Challenges Avances

Consigne

Ces challenges sont ouverts : pas d'indice, pas de structure donnee.

Vous devez concevoir la solution de A a Z.
Ecrivez d'abord votre logique en commentaires avant de coder.
Plusieurs solutions sont possibles -- ce qui compte c'est que ca fonctionne.

Exercice A1 -- Anagrammes | Avance | 25 min

On dit que deux mots sont des anagrammes s'ils contiennent exactement les memes lettres (dans un ordre different).

Implementez :

- sontAnagrammes(mot1, mot2) : retourne true si les deux mots sont anagrammes
- grouperAnagrammes(mots) : regroupe ensemble les mots qui sont anagrammes entre eux

Code	Resultat attendu
`sontAnagrammes('listen','silent')`	`=> true`
`sontAnagrammes('hello','world')`	`=> false`
`sontAnagrammes('Rame','Mare')`	`=> true (insensible a la casse)`
`grouperAnagrammes(['ate','eat','tea','tan','nat','bat'])`	`=> [['ate','eat','tea'],['tan','nat'],['bat']]`

Contrainte
Pour grouperAnagrammes : chaque groupe ne doit apparaitre qu'une seule fois dans le resultat.
L'ordre des groupes n'est pas important.

Exercice A2 -- Sous-tableaux consecutifs | Avance | 25 min

Implementez maxSommeConsecutive(tableau, k) qui trouve la sous-sequence consecutive de longueur k ayant la plus grande somme.

Code	Resultat attendu
`maxSommeConsecutive([2,1,5,1,3,2], 3)`	`=> 9 (sous-tableau : [5,1,3])`
`maxSommeConsecutive([1,4,2,9,7,3,8], 3)`	`=> 20 (sous-tableau : [9,7,4] non... cherchez !)`
`maxSommeConsecutive([1,2,3,4,5], 2)`	`=> 9 (sous-tableau : [4,5])`

BONUS : retournez aussi le sous-tableau lui-meme, pas seulement la somme.

Piste de reflexion
Solution naive : toutes les fenetres possibles avec deux boucles -- O(n*k).
Solution optimale : fenetre glissante, O(n). Comment l'implementer avec reduce ?

Exercice A3 -- Pipeline de transformations | Avance | 30 min

Implementez une fonction pipeline(tableau, ...transformations) qui applique une serie de transformations en chaine sur un tableau.

Chaque transformation est une fonction qui prend un tableau et retourne un tableau. Le resultat de chaque etape devient l'entree de la suivante.

```javascript
// Exemple d'utilisation :
const doublerImpairs = tab => tab.map(n => n % 2 !== 0 ? n * 2 : n);
const garderGrands  = tab => tab.filter(n => n > 5);
const trier         = tab => [...tab].sort((a,b) => a - b);

pipeline([1,2,3,4,5], doublerImpairs, garderGrands, trier);
// etape 1 doublerImpairs : [2, 2, 6, 4, 10]
// etape 2 garderGrands  : [6, 10]
// etape 3 trier          : [6, 10]
// => [6, 10]
```

```javascript
function pipeline(tableau, ...transformations) {
    // Votre code ici
}
```

Une fois pipeline() implemente, construisez le pipeline suivant sur ce tableau de commandes :

```javascript
const commandes = [
    { id:1, client:'Alice', montant:120, statut:'livree'  },
    { id:2, client:'Bob',   montant:45,  statut:'annulee' },
    { id:3, client:'Alice', montant:300, statut:'livree'  },
    { id:4, client:'Clara', montant:80,  statut:'livree'  },
    { id:5, client:'Bob',   montant:200, statut:'livree'  },
    { id:6, client:'Clara', montant:15,  statut:'annulee' },
];
```

Le pipeline doit :

1. Garder uniquement les commandes livrees
2. Garder uniquement celles de plus de 50 EUR
3. Les trier par montant decroissant
4. Extraire uniquement { client, montant } de chaque commande

Resultat attendu
[
{ client:'Alice', montant:300 },
{ client:'Bob', montant:200 },
{ client:'Alice', montant:120 },
{ client:'Clara', montant:80 },
]

Recapitulatif des Methodes

Methode	Role	Mute le tableau ?	Retourne
`push()`	Ajoute a la fin	Oui	Nouvelle longueur
`pop()`	Retire la fin	Oui	Element retire
`shift()`	Retire le debut	Oui	Element retire
`unshift()`	Ajoute au debut	Oui	Nouvelle longueur
`splice()`	Insere / retire	Oui	Elements retires
`forEach()`	Parcourt	Non	undefined
`map()`	Transforme	Non	Nouveau tableau
`filter()`	Filtre	Non	Nouveau tableau
`find()`	1er element trouve	Non	Element ou undefined
`findIndex()`	Index du 1er trouve	Non	Index ou -1
`reduce()`	Agrege	Non	Valeur unique
`sort()`	Trie	Oui	Tableau trie

Ressources recommandees
MDN Web Docs (tableaux) : https://developer.mozilla.org/fr/docs/Web/JavaScript/Reference/Global_Objects/Array
JavaScript.info (methodes) : https://fr.javascript.info/array-methods
Codewars (katas JS niveau 8/7) : https://www.codewars.com
FreeCodeCamp (algorithmes JS) : https://www.freecodecamp.org/learn

Bon courage ! -- Said Aabilla, Formateur DWWM